PARTE 2

Decomposição, escopo e passagem por valor

Organizando responsabilidades e entendendo onde os dados existem

Aluno(a):	Antonio Maia de Jesus Chaves Neto	**Turma:**	Ciência da Computação_Noturno
Data:	27/09/2026	**Professor:**	Diego Abreu

Objetivos

- Decompor um problema em funções com responsabilidades claras.
- Distinguir escopo local, escopo de bloco e variável global.
- Compreender conceitualmente a passagem de argumentos por valor em C.

Atividade 1 - Sistema acadêmico decomposto

Leia três notas, calcule a média e informe a situação. Em vez de colocar toda a lógica dentro de main, use as funções abaixo.

```
double calcularMedia(double a, double b, double c) {
    return (a + b + c) / 3.0;
}

void mostrarSituacao(double media) {
    if (media >= 7.0) {
        printf("Aprovado\n");
    } else {
        printf("Reprovado\n");
    }
}
```

No main, faça apenas a entrada das notas, a chamada de calcularMedia(), a impressão da média e a chamada de mostrarSituacao().

Pergunta de projeto

Qual responsabilidade pertence a cada função? Há alguma responsabilidade que ainda está diretamente em main?

Ao analisar as duas funções isoladamente, divide claramente as responsabilidades de cada uma seguindo boas práticas de programação (como a separação de interesses):

1. responsabilidade de cada função

calcularMedia*(double a, double b, double c)*

A responsabilidade é o processamento matemático/lógica de negócio.

O único objetivo desta função é receber os dados numéricos, aplicar a fórmula da média aritmética e devolver o resultado puro. Ela não funciona para o resto do programa, não sabe quem digitou as notas e não se importa em como o resultado será impresso na tela.

mostrarSituacao(*double media*)

A responsabilidade é avaliação de regra e apresentação.

Esta função aplica a regra de negócio relacionada ao critério de aprovação (o limite de 7.0) e decide qual mensagem textual (aprovado ou reprovado) deve ser comunicada ao usuário. Ela cuida da camada de saída de dados textual.

2. o que ainda fica diretamente na main?

Com base na estrutura que construímos no exemplo anterior, a função *main* deixa de acumular a lógica pesada e passa a assumir dois papéis fundamentais:

A orquestração controla o fluxo de execução, chamando primeiro a função de cálculo, pegando o resultado e passando para a função de situação.

Interface de usuário (*I/O* - entrada e saída) lida diretamente com o mundo externo, ou seja, capturar o que o usuário digita no teclado (*scanf*) e exibir os *prompts* iniciais na tela (*printf*).

Essa separação é excelente ao tirar os cálculos e as regras de validação de dentro da main, o seu código ganha modularidade. Se no futuro você decidir que a média agora considera 4 notas em vez de 3, ou que a nota de corte mudou para 6.0. Alterando apenas a função específica responsável por isso, sem risco de quebrar o restante do programa.

om/#

Run | Debug | Stop | Share | Save | { } Beautify

main.c

```c
#include <stdio.h>

// Função para calcular a média de 3 notas

double calcularMedia(double a, double b, double c) {
    return (a + b + c) / 3.0;
}
// Função para mostrar a situação do aluno com base na média
void mostrarSituacao(double media) {
    if (media >= 7.0) {
        printf("Aprovado\n");
    } else {
        printf("Reprovado\n");
    }
}
int main() {
    double nota1, nota2, nota3;
    printf("Digite tres notas: ");
    scanf("%lf %lf %lf", &nota1, &nota2, &nota3);
    // Chamando as funções que você criou
    double mediaFinal = calcularMedia(nota1, nota2, nota3);

    printf("Media: %.2lf - ", mediaFinal);
    mostrarSituacao(mediaFinal);
    return 0;
}
```

```
Digite tres notas: 4 5 6
Media: 5.00 - Reprovado

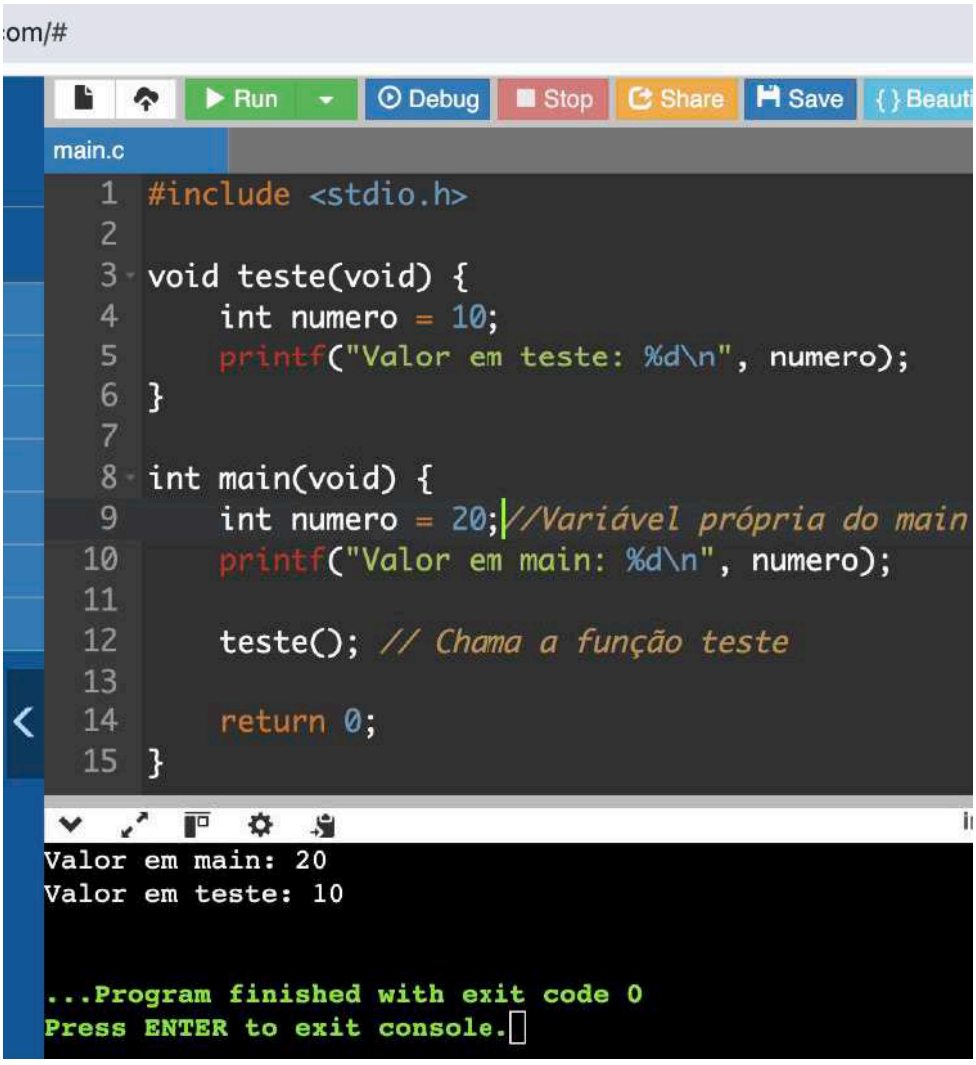

...Program finished with exit code 0
Press ENTER to exit console.
```

Atividade 2 - Escopo local

```
void teste(void) {
    int numero = 10;
    printf("%d\n", numero);
}

int main(void) {
    printf("%d\n", numero);
    return 0;
}
```

Tente compilar. O segundo printf não consegue acessar numero, pois numero é local à função teste().



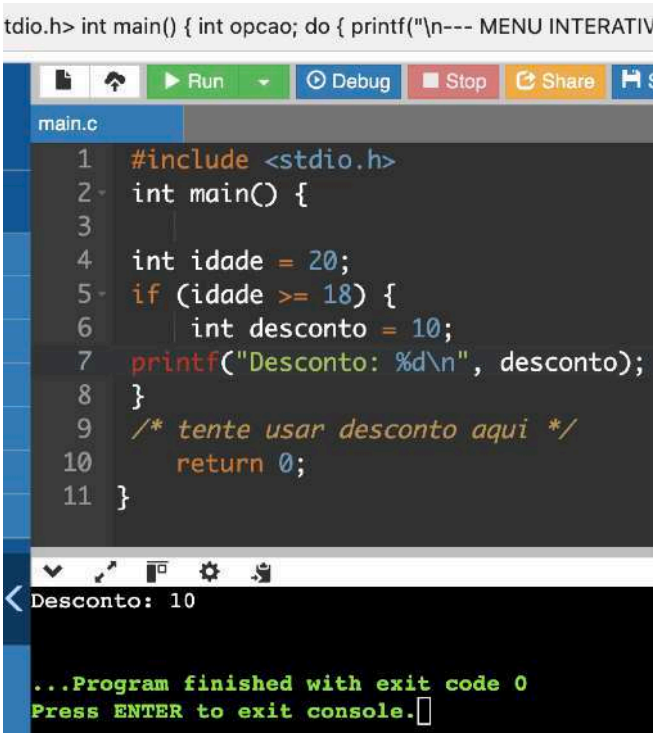
> **Regra prática:** mantenha os dados no menor escopo necessário.

Atividade 3 - Escopo de bloco

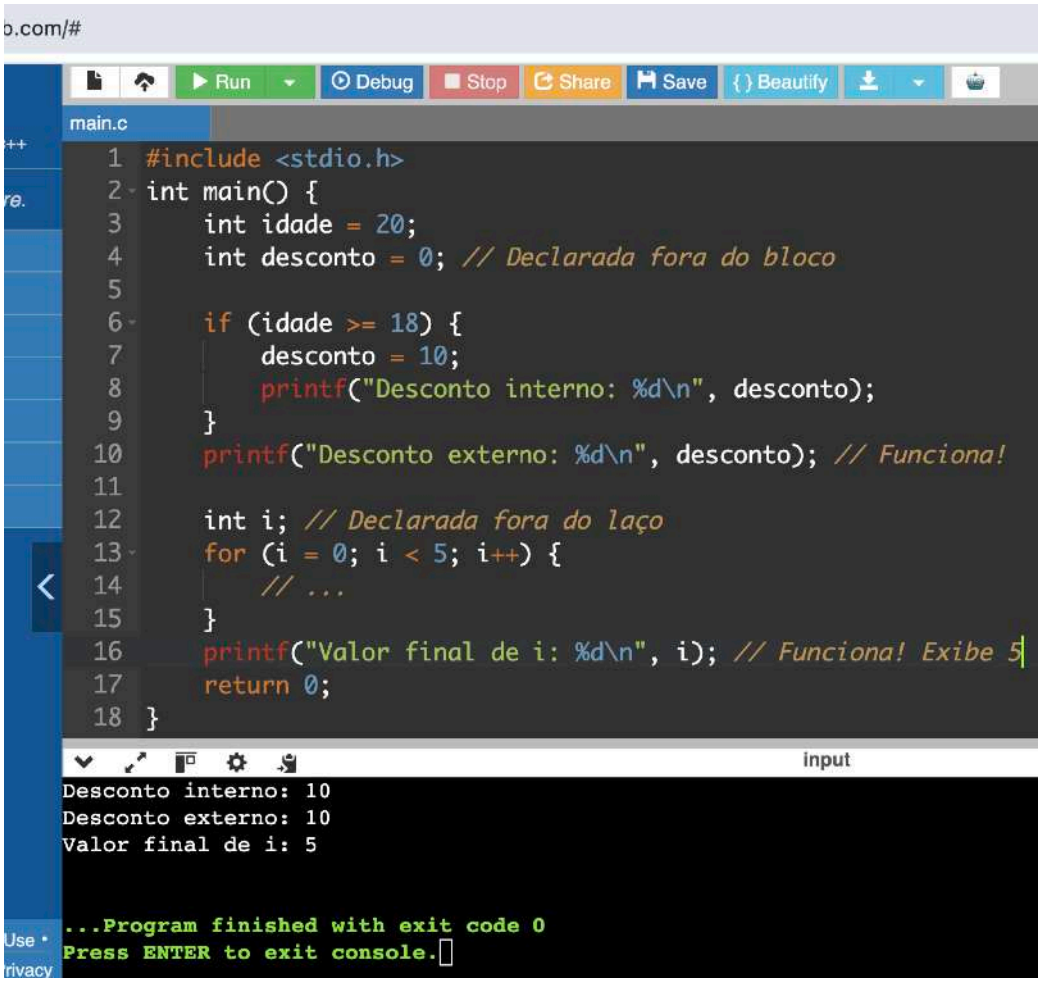
```
int idade = 20;

if (idade >= 18) {
   int desconto = 10;
   printf("Desconto: %d\n", desconto);
}

/* tente usar desconto aqui */
```



Teste também um for com int i declarado dentro do próprio cabeçalho e tente usar i após o laço.



Atividade 4 - Local versus global

Compare as duas ideias:

```
int calcularDobro(int numero) {
   return numero * 2;
}
```

```
int numeroGlobal;

int calcularDobro(void) {
   return numeroGlobal * 2;
}
```

1. Qual versão deixa mais explícito de onde vem o valor usado no cálculo?

A primeira versão deixa mais explícito de onde vem o valor usado no cálculo, pois o número é recebido diretamente pelo parâmetro numero (calcularDobro(int numero)). A função depende apenas do argumento recebido.
A segunda versão depende de um estado externo à função, pois utiliza a variável global numeroGlobal, que não é passada como argumento. Ela depende de uma variável global externa (calcularDobro(void)), cujo valor pode ser alterado por outras partes do programa.

2. Qual versão depende de um estado externo à função?

> **Atenção:** variáveis globais existem e podem ser úteis, mas para este curso prefira parâmetros, retornos e variáveis locais sempre que possível.

A segunda versão depende de um estado externo à função, utilizando uma variável global numeroGlobal, que não é passada como argumento. Ela depende de uma variável global externa (calcularDobro(void)).

Atividade 5 - Passagem por valor

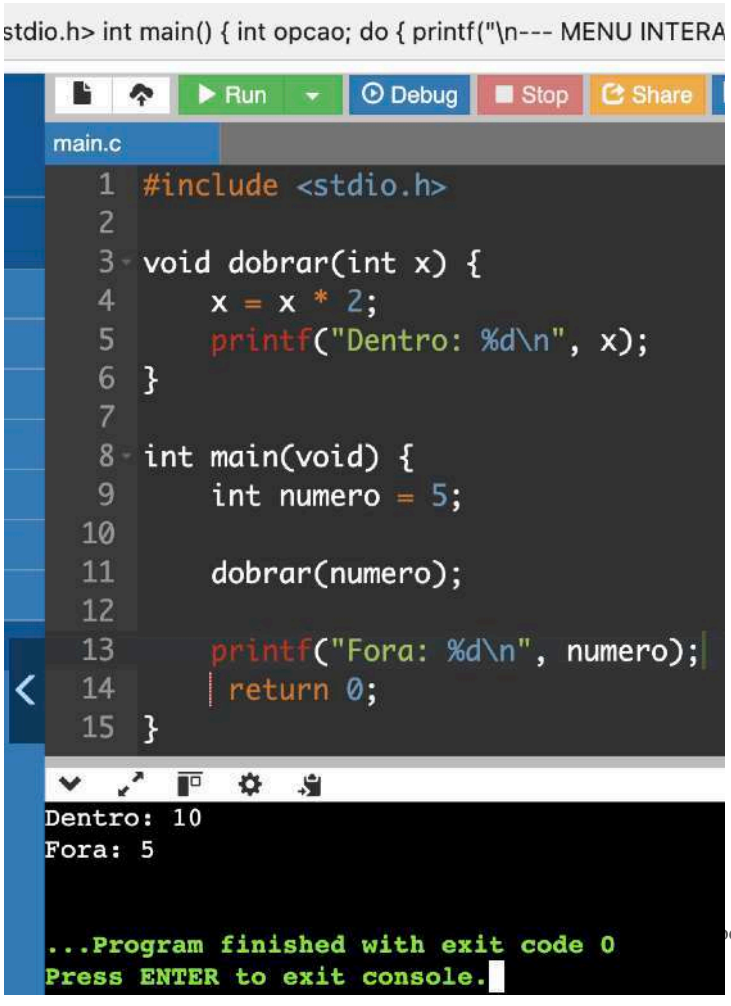
Preveja o resultado antes de executar.

```
void dobrar(int x) {
    x = x * 2;
    printf("Dentro: %d\n", x);
}

int main(void) {
    int numero = 5;
    dobrar(numero);
    printf("Fora: %d\n", numero);
    return 0;
}
```

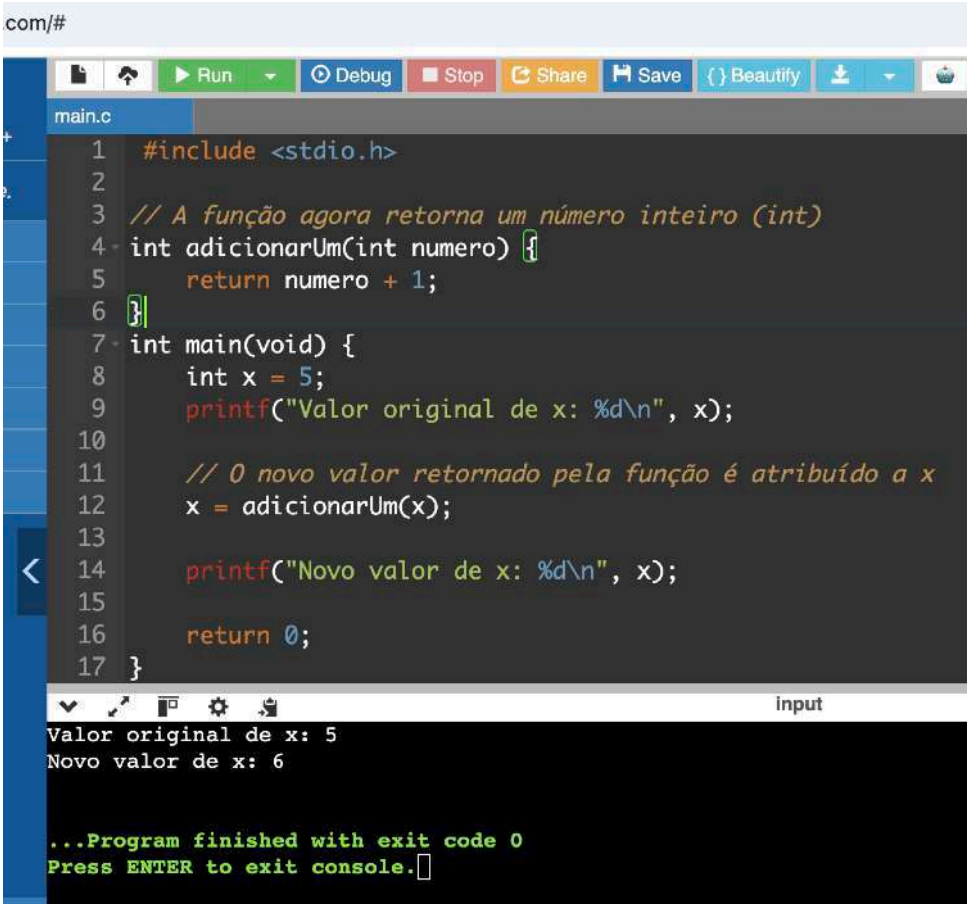
Saída esperada: dentro da função x vale 10, mas numero continua 5. A função recebeu uma cópia do valor.

> **Gancho para ponteiros:** se argumentos comuns são passados por valor, como scanf consegue alterar idade? Porque enviamos &idade, isto é, um endereço. Essa ideia será aprofundada posteriormente.



Desafio - Corrigindo a intenção

A função abaixo não altera x em main. Reescreva o programa para que o novo valor seja obtido por retorno.




```
void adicionarUm(int numero) {
   numero++;
}
```

3. Por que uma variável local de uma função não pode ser usada automaticamente em outra?

Resposta: Uma variável local não pode ser usada em outra função devido ao escopo de variáveis e ao gerenciamento de memória ram em linguagens de programação C. As principais razões são:

1. Limitação de escopo (visibilidade)

O escopo de uma variável local é restrito ao bloco de código onde ela foi declarada (entre as chaves { }). O compilador só reconhece o nome daquela variável dentro da própria função. Para qualquer outra função, essa variável simplesmente não existe.

2. Tempo de vida e memória (pilha/stack)

As variáveis locais são criadas na memória (na região chamada stack ou pilha) no momento em que a função é chamada e destruídas automaticamente assim que a função termina a sua execução. Portanto, quando outra função estiver rodando, a memória da função anterior pode já ter sido liberada ou sobrescrita.

3. Modulariedade e independência

Essa limitação é fundamental para evitar que uma função altere acidentalmente os dados de outra. Ela permite também que diferentes funções usem variáveis com o mesmo nome (como i, x ou opção) sem que haja conflito entre elas.

4. Qual é a diferença entre escopo de função e escopo de bloco?

Resposta:

Os escopos de bloco são mais granulares. Em uma função, você tem um escopo de função (não contando funções aninhadas) e pode ter múltiplos escopos de bloco dentro desse escopo de função. Se você tiver um *var* em qualquer um dos escopos de bloco, a declaração é escopada para a função, não para o bloco. *let* e const (e *class* e *using* e *function* em modo estrito) irão ser escopados para aqueles blocos específicos.

Em linguagem C, o escopo de função aplica-se exclusivamente a rótulos (*labels* de comandos como goto), enquanto o escopo de bloco restringe o uso de variáveis e outros identificadores ao interior de um par de chaves {} específico.

Escopo de Função (*Function Scope*) se aplica apenas para rótulos de desvio (*labels*), como os usados pelo comando goto.

O funcionamento se baseia num rótulo declarado em qualquer lugar de uma função é visível em toda a extensão dessa mesma função, independentemente de estar dentro de blocos aninhados ou chaves mais internas. Ele não pode ser acessado de fora da função em que foi criado.

Escopo de Bloco (Block Scope)

Ele se aplica em variáveis locais e parâmetros funcionando com uma variável declarada dentro de um bloco delimitado por chaves {} (como o corpo de uma função, um laço for ou uma condição if) só existe e pode ser acessada de dentro daquele bloco específico. Assim que o fluxo de execução sai do bloco, a variável deixa de existir. Em linguagem C, todo escopo de função também forma um bloco principal, mas o escopo de bloco permite delimitações ainda mais granulares.

5. Por que numero continua 5 após a chamada de dobrar(numero)?

Resposta:

Foi porque x sempre recebe 5;

```
int numero = 5;
   dobrar(numero);
```

Entrega da parte

- Sistema acadêmico decomposto.
- Experimentos de escopo compilados e analisados.
- Versão corrigida de adicionar Um usando retorno.

